MODUL PEMBELAJARAN

Analisis & Desain Perangkat Lunak

Pertemuan 1: Konsep Rekayasa Perangkat Lunak dan SDLC

Program Studi S1 Teknik Informatika · Fakultas Teknik · Universitas Muhammadiyah Cirebon

Mata kuliah	Analisis & Desain Perangkat Lunak (3 SKS) · Semester 4
Pertemuan	1 · TM 2×50′; PT 2×60′; BM 2×60′
Sub-CPMK	**Sub-CPMK-1** — Menjelaskan konsep SDLC serta membandingkan model Waterfall, Prototyping, Spiral, dan Agile.
CPMK / CPL	CPMK-1 → CPL04
Studi kasus	SIPUS — Sistem Informasi Perpustakaan Kampus (dipakai di seluruh modul)
Dosen pengampu	(nama dosen pengampu)

Tujuan Pembelajaran

Setelah mempelajari modul ini, mahasiswa mampu:

1. Menjelaskan pengertian perangkat lunak dan rekayasa perangkat lunak (RPL).
2. Menguraikan tahapan Software Development Life Cycle (SDLC) beserta luaran tiap tahap.
3. Membandingkan model proses Waterfall, Prototyping, Spiral, dan Scrum.
4. Memilih model proses yang sesuai untuk suatu kasus beserta alasannya.

Indikator Penilaian

- Ketepatan menjelaskan tahapan SDLC.
- Ketepatan membandingkan kelebihan dan kekurangan model proses.
- Kesesuaian pilihan model proses dengan karakteristik kasus.

1. Perangkat Lunak dan Rekayasa Perangkat Lunak

Perangkat lunak bukan hanya kode program. Perangkat lunak mencakup program yang dapat dijalankan, struktur data yang diolahnya, serta dokumentasi yang menjelaskan cara membangun, memakai, dan memeliharanya. Karena itu, kualitas perangkat lunak ditentukan sejak tahap awal: seberapa baik kebutuhan dipahami dan seberapa baik solusi dirancang.

Rekayasa perangkat lunak (RPL) adalah penerapan pendekatan yang sistematis, disiplin, dan terukur dalam pengembangan, operasi, dan pemeliharaan perangkat lunak. Kata

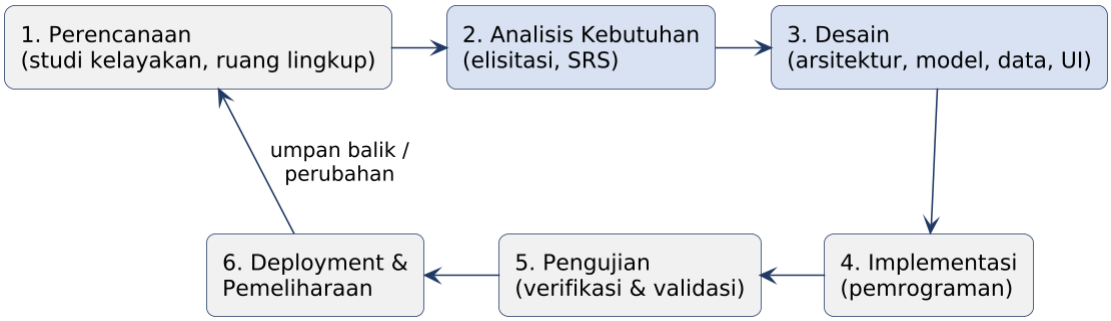
kuncinya adalah *sistematis*: pekerjaan dibagi dalam tahapan yang jelas, setiap tahap menghasilkan artefak yang dapat diperiksa, dan keputusan teknis dapat dipertanggungjawabkan.

Mengapa perlu "rekayasa"? Proyek perangkat lunak sering gagal bukan karena programmer tidak mampu menulis kode, melainkan karena kebutuhan yang salah dipahami, ruang lingkup yang terus berubah, dan rancangan yang tidak siap menghadapi perubahan. Mata kuliah ini berfokus pada dua kegiatan yang paling menentukan keberhasilan proyek, yaitu **analisis** (memahami masalah dan kebutuhan) dan **desain** (merancang solusi sebelum dibangun).

> **Karakteristik perangkat lunak yang baik**
>
> ▪ Benar (correct): melakukan apa yang dibutuhkan pengguna.
> ▪ Andal (reliable) dan aman (secure): bekerja konsisten dan terlindung dari penyalahgunaan.
> ▪ Mudah dipakai (usable) dan efisien.
> ▪ Mudah dipelihara (maintainable): dapat diubah tanpa merusak bagian lain.

2. Software Development Life Cycle (SDLC)

SDLC adalah kerangka tahapan yang dilalui perangkat lunak sejak gagasan muncul hingga sistem digunakan dan dipelihara. Nama dan jumlah tahap dapat berbeda antarbuku, tetapi intinya sama. Gambar berikut menunjukkan tahapan umum SDLC beserta posisi mata kuliah ini.



Gambar 1.1 Tahapan umum SDLC; tahap yang diarsir adalah fokus mata kuliah ini

Tabel 1.1 Tahapan SDLC dan luarannya

Tahap	Kegiatan utama	Luaran (artefak)
Perencanaan	Menentukan tujuan, ruang lingkup, jadwal, biaya, dan kelayakan proyek.	Proposal proyek, studi kelayakan
Analisis	Menggali dan menganalisis kebutuhan pemangku kepentingan.	Dokumen SRS, model use case
Desain	Merancang arsitektur, model kelas, interaksi, basis data, dan antarmuka.	Dokumen SDD, diagram UML, ERD, prototype
Implementasi	Menulis kode sesuai rancangan.	Kode sumber, build
Pengujian	Memastikan sistem benar (verifikasi) dan sesuai kebutuhan (validasi).	Rencana dan laporan uji

Tahap	Kegiatan utama	Luaran (artefak)
Deployment & pemeliharaan	Memasang sistem, melatih pengguna, memperbaiki dan mengembangkan.	Sistem berjalan, catatan perubahan

Biaya memperbaiki kesalahan meningkat tajam semakin lambat kesalahan itu ditemukan. Kesalahan pemahaman kebutuhan yang baru ketahuan saat pengujian bisa memaksa tim mengubah rancangan dan kode sekaligus. Inilah alasan analisis dan desain yang cermat sangat berharga.

3. Model Proses Pengembangan

Model proses menentukan *urutan dan cara* tahapan SDLC dijalankan. Tidak ada model yang terbaik untuk semua proyek; pilihan bergantung pada kejelasan kebutuhan, ukuran dan risiko proyek, serta keterlibatan pengguna.

3.1 Waterfall

Tahapan dijalankan berurutan; tahap berikutnya dimulai setelah tahap sebelumnya selesai dan disetujui. Model ini mudah dikelola dan dokumentasinya lengkap, tetapi kaku terhadap perubahan. Cocok bila kebutuhan sudah stabil dan jelas sejak awal, misalnya sistem yang mengikuti regulasi tetap.

3.2 Prototyping

Tim membuat prototype (tiruan sistem) secara cepat, pengguna mencoba dan memberi masukan, lalu prototype diperbaiki berulang hingga kebutuhan jelas. Cocok bila pengguna sulit menjelaskan keinginannya. Risikonya, pengguna menganggap prototype sudah jadi, dan tim tergoda melanjutkan kode prototype yang kualitasnya rendah.

3.3 Spiral

Pengembangan berjalan dalam putaran (spiral). Setiap putaran melewati empat kuadran: menentukan tujuan, **menganalisis risiko**, membangun dan menguji, lalu merencanakan putaran berikutnya. Kekuatan utamanya adalah manajemen risiko, sehingga cocok untuk proyek besar dan berisiko tinggi. Kelemahannya, model ini mahal dan membutuhkan keahlian analisis risiko.

3.4 Agile dan Scrum

Agile adalah filosofi pengembangan yang mengutamakan kolaborasi dengan pengguna, perangkat lunak yang berfungsi, dan kesiapan menghadapi perubahan. Scrum adalah salah satu kerangka Agile yang paling banyak dipakai. Pekerjaan dibagi ke dalam **sprint** berdurasi 1–4 minggu; setiap sprint menghasilkan bagian produk yang dapat dipakai.

- **Peran:** Product Owner (menentukan prioritas), Scrum Master (memfasilitasi proses), Development Team.
- **Artefak:** Product Backlog (daftar kebutuhan), Sprint Backlog, Increment (hasil sprint).
- **Acara:** Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective.

Agile tidak berarti tanpa analisis dan desain. Analisis tetap dilakukan, tetapi secara bertahap: kebutuhan ditulis sebagai user story dan dirinci menjelang dikerjakan, dan desain disempurnakan setiap sprint.

Tabel 1.2 Perbandingan model proses

Aspek	Waterfall	Prototyping	Spiral	Scrum
Kejelasan kebutuhan	Harus jelas di awal	Belum jelas	Berkembang	Berkembang
Keterlibatan pengguna	Awal dan akhir	Tinggi	Tiap putaran	Tinggi, berkelanjutan
Respons terhadap perubahan	Rendah	Tinggi	Sedang	Tinggi
Fokus utama	Dokumentasi & urutan	Umpan balik pengguna	Risiko	Nilai bisnis tiap sprint
Cocok untuk	Kebutuhan stabil, regulasi	UI baru, pengguna awam	Proyek besar berisiko	Produk yang terus berkembang

4. Memilih Model Proses: Studi Kasus SIPUS

Sepanjang semester, modul ini memakai studi kasus **SIPUS (Sistem Informasi Perpustakaan Kampus)**. Perpustakaan kampus ingin mengganti pencatatan manual dengan sistem yang mendukung katalog daring, peminjaman dan pengembalian, pemesanan buku, perhitungan denda, dan laporan bulanan. Pustakawan belum terbiasa dengan aplikasi, kebutuhan laporan pimpinan masih bisa berubah, dan kampus ingin fitur katalog daring dapat dipakai secepatnya.

Dengan karakteristik tersebut, **Scrum** (atau Agile dengan prototype antarmuka di awal) lebih tepat daripada Waterfall. Alasannya: kebutuhan laporan belum stabil, pengguna perlu melihat sistem untuk memberi masukan, dan katalog daring dapat dirilis pada sprint awal sebagai nilai cepat. Waterfall akan berisiko karena perubahan kebutuhan laporan baru muncul setelah sistem selesai.

Cara berpikir saat memilih model

- Seberapa jelas dan stabil kebutuhan?
- Seberapa besar risiko teknis dan bisnisnya?
- Seberapa sering pengguna dapat dilibatkan?
- Apakah ada tuntutan dokumentasi formal (misalnya kontrak atau regulasi)?

Aktivitas Pembelajaran

Moda	Kegiatan dan Penugasan
Luring	Ceramah interaktif dan diskusi kelas; penjelasan RPS dan kontrak kuliah; pembentukan tim proyek 4–5 orang. Penugasan: tim menentukan mitra studi kasus untuk proyek semester.
Daring (LMS)	Belajar mandiri melalui video dan bacaan di LMS; forum diskusi. Penugasan: kuis K1 (pilihan ganda + esai singkat) dan unggahan perbandingan model proses di forum.

Latihan dan Penugasan

Latihan Mandiri

1. Jelaskan dengan kata-kata sendiri perbedaan antara pemrograman dan rekayasa perangkat lunak.
2. Sebutkan luaran utama tahap analisis dan tahap desain dalam SDLC.
3. Mengapa kesalahan kebutuhan yang ditemukan saat pengujian lebih mahal diperbaiki daripada saat analisis?
4. Sebuah dinas ingin membangun aplikasi pelaporan pajak dengan aturan yang sudah ditetapkan undang-undang. Model proses apa yang paling sesuai? Jelaskan alasannya.

Tugas Tim (luring)

1. Bentuk tim 4–5 orang dan tentukan calon mitra studi kasus (UMKM, amal usaha Muhammadiyah, atau unit kampus).
2. Tuliskan deskripsi singkat masalah mitra (maksimal satu halaman).
3. Pilih model proses yang akan dipakai tim dan berikan tiga alasan berdasarkan karakteristik mitra.

Rangkuman

- RPL adalah pendekatan sistematis, disiplin, dan terukur untuk membangun dan memelihara perangkat lunak.
- SDLC terdiri atas perencanaan, analisis, desain, implementasi, pengujian, serta deployment dan pemeliharaan.
- Waterfall cocok untuk kebutuhan stabil; Prototyping untuk kebutuhan yang belum jelas; Spiral untuk proyek berisiko tinggi; Scrum untuk produk yang berkembang dengan keterlibatan pengguna tinggi.
- Analisis dan desain tetap diperlukan dalam semua model, termasuk Agile; yang berbeda adalah waktu dan kedalamannya.

Daftar Pustaka

[1] Pressman, R. S., & Maxim, B. R. Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.

[2] Sommerville, I. Software Engineering (10th ed.). Pearson.